

Η γλώσσα προγραμματισμού Java

Προσαρμογή από τις σημειώσεις του κ. Ι. Χαμόδρακα (ΕΔΙΠ τμήματος)

Βασικές έννοιες Αντικειμενοστρεφούς Προγραμματισμού (1/4)

- Παράδειγμα από την καθημερινότητα:
 - Ένα αυτοκίνητο είναι ένα **αντικείμενο (object)**
 - Τα **χαρακτηριστικά (attributes)** αυτού του αυτοκινήτου είναι το μοντέλο, ο χρόνος κατασκευής, το χρώμα κλπ.
 - Η συμπεριφορά του αυτοκινήτου καθορίζεται από τις **μεθόδους (methods)** που μπορούν να του εφαρμοστούν (π.χ. ξεκίνα, σταμάτα, στρίψε δεξιά, κλπ)
 - Για να εφαρμοστεί μια μέθοδος πρέπει να δοθεί ένα κατάλληλο μήνυμα (**message**) π.χ. το στρίψιμο του τιμονιού.

Βασικές Έννοιες Αντικειμενοστρεφούς Προγραμματισμού (2/4)

- Όλα τα αυτοκίνητα έχουν παρόμοιες ιδιότητες (**properties**) και μπορούν να καταταγούν στην αφηρημένη κλάση αντικειμένων (**class**) «αυτοκίνητο». Τα συγκεκριμένα υπαρκτά αυτοκίνητα ονομάζονται στιγμιότυπα (**instance**) της κλάσης.
- Οι κλάσεις μπορούν να κληρονομούν κοινές ιδιότητες από άλλες κλάσεις (**inheritance**). Π.χ. μπορεί να έχουμε την υπερκλάση των μεταφορικών μέσων, οι οποία έχει κάποιες ιδιότητες που είναι κοινές τόσο στα αυτοκίνητα, όσο και στα τρένα ή τα αεροπλάνα

Βασικές Έννοιες Αντικειμενοστρεφούς Προγραμματισμού (3/4)

- Αντικείμενο λογισμικού είναι ένα κομμάτι δεδομένων που περιβάλλεται από κώδικα.
- Δηλαδή πρόσβαση στα δεδομένα του αντικειμένου μπορεί να γίνει μέσω της κλήσης κάποιας μεθόδου. Ο κώδικας του αντικειμένου αποτελείται από τις μεθόδους του αντικειμένου. Τα δεδομένα έχουν κάποια συγκεκριμένα χαρακτηριστικά (ή πεδία, π.χ. όνομα, διεύθυνση), και οι μέθοδοι χρησιμοποιούνται προκειμένου να εκτελεστούν κάποιες λειτουργίες πάνω σε αυτά τα πεδία.
- Τα αντικείμενα επικοινωνούν μεταξύ τους μέσω μηνυμάτων.
- Ένα μήνυμα χρησιμοποιείται για την κλήση μίας μεθόδου

Βασικές Έννοιες Αντικειμενοστρεφούς Προγραμματισμού (4/4)

- Μια συλλογή αντικειμένων με κοινές ιδιότητες αποτελεί μια κλάση αντικειμένων. Ένα αντικείμενο είναι στιγμιότυπο μιας κλάσης.
- Τα αντικείμενα μπορούν να κληρονομούν ιδιότητες από άλλα αντικείμενα. Τεχνικά, κληρονομικότητα σημαίνει τη δυνατότητα δημιουργίας υποκλάσεων μιας κλάσης.
- Οι βασικές ιδιότητες επομένως των αντικειμένων είναι:
 - ▣ Ενθυλάκωση (Encapsulation): Ο περιορισμός της πρόσβασης στα δεδομένα του αντικειμένου μέσω καλά ορισμένων διεπαφών.
 - ▣ Αφαίρεση (Abstraction): Η ικανότητα ομαδοποίησης συσχετιζόμενων οντοτήτων βάσει των κοινών ιδιοτήτων τους.
 - ▣ Πολυμορφισμός: Η δυνατότητα επικάλυψης και τομής μεταξύ των αφαιρέσεων (π.χ. μέσω της κληρονομικότητας)

Αντικειμενοστρεφής Προγραμματισμός (Object-Oriented Programming)

- Το πρόγραμμα ουσιαστικά μοντελοποιεί έναν “κόσμο” ενεργών αντικειμένων (objects)
- Ένα αντικείμενο έχει την δική του ιδιωτική μνήμη η οποία μπορεί να περιέχει άλλα αντικείμενα.
- Ένα αντικείμενο περιέχει τις μεθόδους που προσπελάζουν και μεταβάλλουν την μνήμη του.

Αντικειμενοστρεφής προγραμματισμός (συνέχεια)

- Μία μέθοδος μπορεί να αλλάζει την κατάσταση του αντικειμένου, να στέλνει μηνύματα και να επικοινωνεί με άλλα αντικείμενα καθώς και να δημιουργεί νέα αντικείμενα.
- Κάθε αντικείμενο ανήκει σε μία συγκεκριμένη κλάση (στιγμιότυπο κλάσης) η οποία καθορίζει και την λειτουργικότητά του.
- Ένας προγραμματιστής δημιουργεί μία εφαρμογή ορίζοντας κατάλληλες κλάσεις.

Βασικό χαρακτηριστικό του Αντικειμενοστρεφούς Προγραμματισμού:

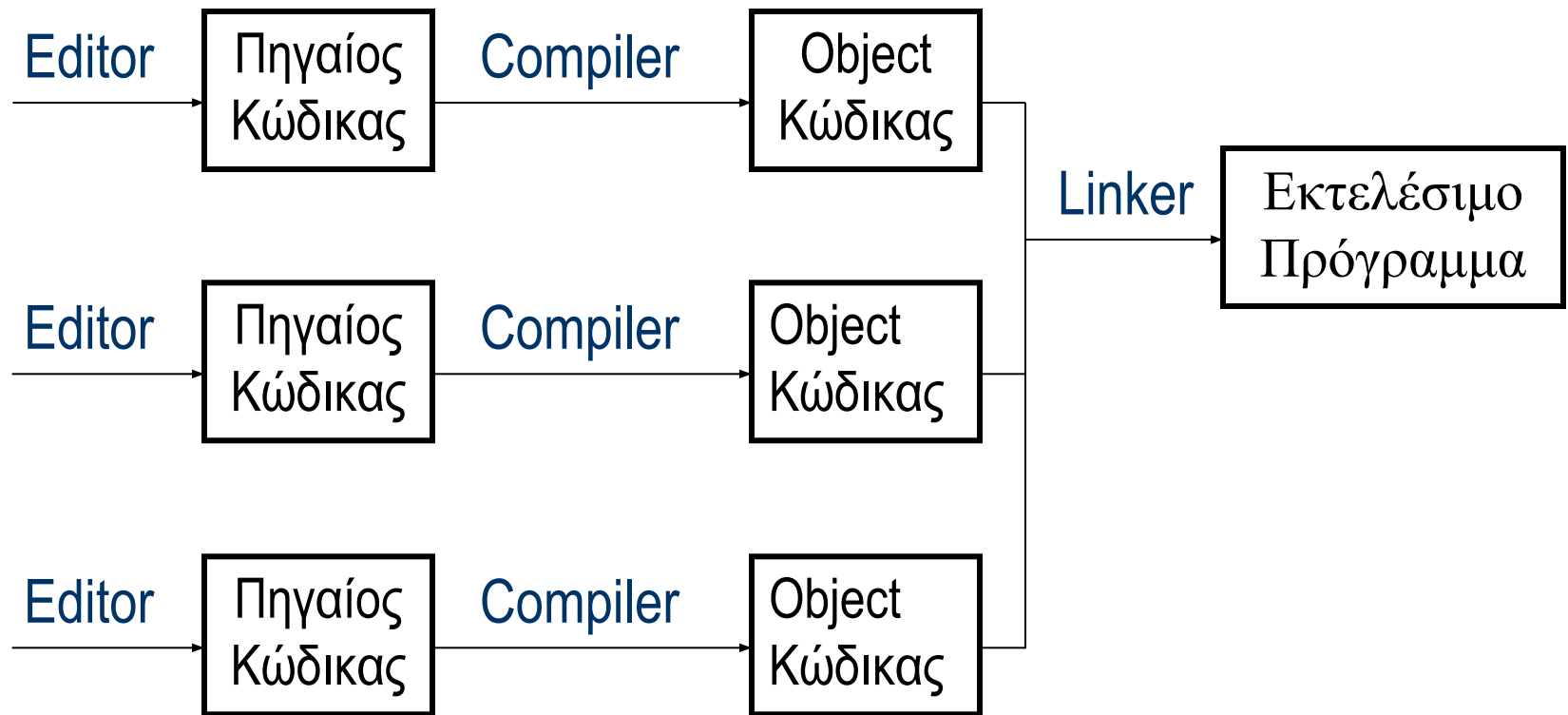
- Επαναχρησιμοποίηση (Reusability): Οι αντικειμενοστρεφείς γλώσσες είναι πιστές στο δόγμα “δεν ανακαλύπτουμε ξανά τον τροχό”. Τα προγράμματα γράφονται σπανίως από την αρχή. Συνηθέστερα, ο κώδικας που έχει ήδη γραφεί επαναχρησιμοποιείται με την μορφή βιβλιοθηκών (libraries).

Βασική έννοια του αντικειμενοστρεφούς προγραμματισμού:

- Κληρονομικότητα (Inheritance): Μία κλάση επεκτείνει την λειτουργικότητα της υπερκείμενής της κλάσης (*superclass*). Τα αντικείμενα μιας κλάσης κληρονομούν τα χαρακτηριστικά/μέλη της *superclass* και μπορούν να τα ορίζουν εκ νέου ή να προσθέτουν νέα χαρακτηριστικά/μέλη.

Ο κύκλος ανάπτυξης με χρήση compiler:

Edit-Compile-Link-Run



Ο κύκλος ανάπτυξης με χρήση interpreter:

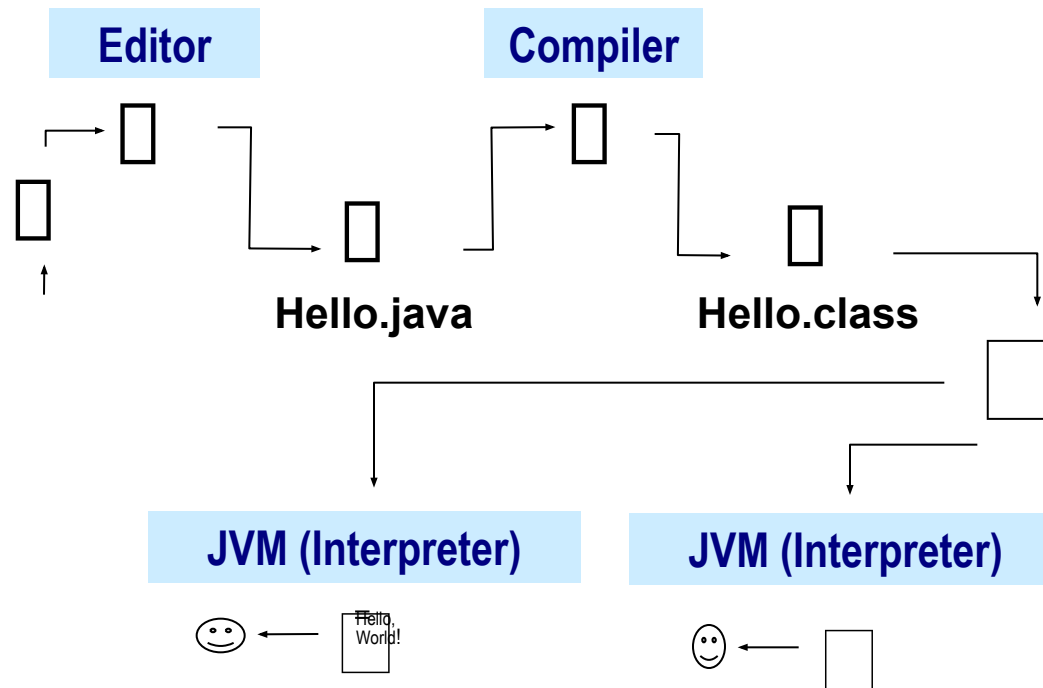
Edit-Run



Η υβριδική προσέγγιση της Java: Compiler + Interpreter

- Ο compiler της Java μετατρέπει τον πηγαίο κώδικα της Java σε εντολές κατανοητές από την JVM (Java Virtual Machine).
- JVM: εικονικός υπολογιστής με ένα σύνολο εντολών CPU που είναι ο Ε.Κ.Π. των συνηθισμένων CPU
- Αυτές οι εντολές οι οποίες καλούνται *bytecodes*, είναι ίδιες για κάθε υπολογιστή ή λειτουργικό σύστημα.
- Ένας ειδικός για κάθε CPU Java interpreter μετατρέπει τα *bytecodes* σε κώδικα εκτελέσιμο στην συγκεκριμένη CPU.

Ο κύκλος ανάπτυξης ενός Java προγράμματος



Γιατί η χρήση του ενδιάμεσου κώδικα (Bytecodes);

- Εκτέλεση ανεξαρτήτως πλατφόρμας
- Φορτώνεται από το διαδίκτυο γρηγορότερα απ' ότι ο πηγαίος κώδικας
- Ο Interpreter είναι τώρα γρηγορότερος και μικρότερος σε μέγεθος
- Ο Interpreter έχει την δυνατότητα να εκτελεί περισσότερους ελέγχους ασφαλείας

JDK — Java Development Kit

- **javac**

- Ο compiler της java

- **java**

- Ο interpreter της java

- **appletviewer**

- ελέγχει τα applets χωρίς την χρήση browser

- **jdb**

- Ο java debugger

- **javadoc**

- παράγει από τον κώδικα τεκμηρίωση σε μορφή html

- **jar**

- συμπιέζει ένα σύνολο από συσχετιζόμενες κλάσεις σε ένα αρχείο (jar αρχείο) για ευκολότερη μεταφορά

Όλα τα παραπάνω είναι command-line εργαλεία

JDK (συνέχεια)

- Διατίθεται δωρεάν από την Oracle
- Όλη η τεκμηρίωση μπορεί να βρεθεί στην διεύθυνση:

<http://docs.oracle.com/javase/8/docs/>

- Υπάρχουν πάρα πολλές πηγές για την Java στο διαδίκτυο

Java IDE

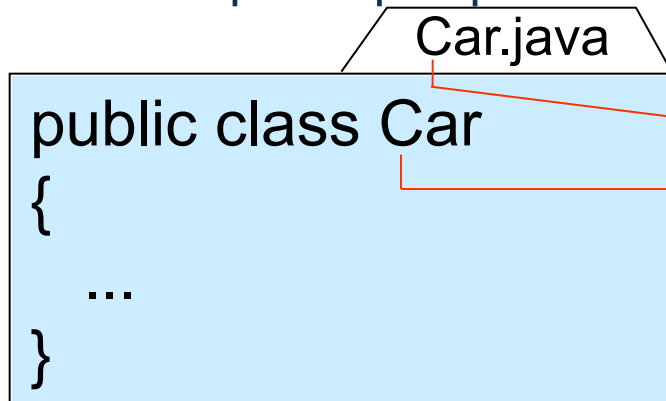
- Γραφικό περιβάλλον (GUI) για το JDK
- Περιλαμβάνει editor, javac, java, appletviewer, debugger, και άλλα εργαλεία:
 - ειδικό editor για την σύνταξη της Java
 - γρήγορη μεταφορά στο σημείο του προγράμματος που έχει συντακτικό λάθος
- Συνήθως εγκαθίσταται πάνω από το JDK ανεξάρτητα από αυτό
- Πλατφόρμα μαθήματος: Netbeans 8 IDE

Java class

- Κάθε πρόγραμμα Java περιέχει τουλάχιστον μία class
- Κάθε class έχει όνομα (identifier), μέλη (members) ενώ είναι δυνατόν να περιέχει σχόλια
- Τα μέλη μπορεί να είναι πεδία (μεταβλητές) και μέθοδοι (συναρτήσεις).
- Τα πεδία συνθέτουν την κατάσταση (state) της κλάσης.

Οι κλάσεις και τα αρχεία πηγαίου κώδικα

- Κάθε κλάση συνήθως αποθηκεύεται σε ένα χωριστό αρχείο
- Το όνομα του αρχείου πρέπει να συμπίπτει με το όνομα της κλάσης με επέκταση .java όταν η κλάση ορίζεται ως public
- Το όνομα δε μπορεί να είναι λέξη κλειδί της java



Γίνεται η σύμβαση ότι το όνομα της κλάσης (και συνεπώς και του αρχείου) αρχίζει με κεφαλαίο γράμμα.

(Στην Java όλα τα ονόματα είναι case-sensitive.)

Κλάσεις Αντικειμένων (1)

- Μια κλάση αντικειμένων στον αντικειμενοστρεφή προγραμματισμό (OOP) αποτελεί έναν νέο τύπο δεδομένων. Από τη στιγμή που έχει οριστεί ένας καινούριος τύπος, μπορούν να δημιουργηθούν αντικείμενα αυτού του τύπου. Η «διεπαφή» του τύπου ορίζει τις λειτουργίες που μπορεί να εκτελέσει το object αν κληθεί.
- Π.χ. Class Name: Light
 - Διεπαφή: on(), off(), brighten(), dim() (μέθοδοι του αντικειμένου)
- Εκτός από τι μεθόδους, η κλάση μπορεί να περιέχει και άλλα μέλη που περιλαμβάνουν τα δεδομένα (πεδία / χαρακτηριστικά) κάποιου τύπου, άλλες κλάσεις (εσωτερικές κλάσεις), κλπ.

Κλάσεις Αντικειμένων (2)

- Η δήλωση `Light lt` σημαίνει ότι η μεταβλητή `lt` αναπαριστά ένα αντικείμενο τύπου `Light`. Θα μπορούσαμε απλά να γράψουμε:
`Light lt;`
- Για να δημιουργηθεί όμως το object δεν αρκεί αυτό, καλείται μια ειδική διαδικασία που ονομάζεται `constructor` με το πρόθεμα `new`:
`lt = new Light();`
- Στη συνέχεια για να σταλεί ένα μήνυμα στο αντικείμενο χρησιμοποιείται το `handle lt` ακολουθούμενο από μια τελεία και την αντίστοιχη μέθοδο.
- Αν λοιπόν θεωρήσουμε ότι με κάποιο τρόπο έχει οριστεί αυτός ο τύπος, για να τον χρησιμοποιήσουμε γράφουμε:
 - `Light lt = new Light();`
`lt.on();`

Κλάσεις Αντικειμένων (3)

- Ο χειρισμός των αντικειμένων γίνεται μέσω των handles (αναφορών/μεταβλητών). Π.χ. `Object o`; `String s`;
- Με αυτόν τον τρόπο δηλώνεται ότι το `o` είναι αναφορά σε αντικείμενο τύπου `Object`. Έτσι δεν δημιουργείται το αντικείμενο αλλά μόνο η αναφορά σε αυτό. Δηλαδή εκχωρείται μνήμη για την αποθήκευση της αναφοράς
- Δεν υπάρχει ανάγκη ρητής καταστροφής των δημιουργημένων αντικειμένων, διαγραφής τους δηλαδή από τη μνήμη, όπως π.χ. πρέπει να γίνεται στην C για να μην υπάρχουν `memory leaks`. Αυτό το αναλαμβάνει ο `garbage collector` της Java (runtime)

Βιβλιοθήκες (Libraries)

- Τα προγράμματα της Java δεν γράφονται συνήθως από το μηδέν.
- Υπάρχουν εκατοντάδες βιβλιοθήκες κλάσεων για όλες τις περιπτώσεις.
- Οι σχετιζόμενες κλάσεις οργανώνονται σε packages.

Για παράδειγμα:

`java.util` — miscellaneous utility classes

`java.awt` — windowing and graphics toolkit

`javax.swing` — GUI development package

Βιβλιοθήκες (Libraries)(2)

- Οι κλάσεις οργανώνονται σε πακέτα (packages). Π.χ.:
 - `package lectures;`
- Για να χρησιμοποιηθούν κλάσεις ενός διαφορετικού πακέτου εντός μιας κλάσης, θα πρέπει να δηλωθούν με το keyword `import`
`import packagename.classname;`
`import packagename.*;`
- Για packages που έχουμε δημιουργήσει, θα πρέπει να ορίσουμε και τη θέση τους στην παράμετρο `CLASSPATH` (μεταβλητή συστήματος ή ορισμένης μέσω της γραμμής εντολών) ώστε να είναι δυνατό να βρεθούν από τον `compiler`.

import

- Το πλήρες όνομα των κλάσεων βιβλιοθηκών περιλαμβάνει και το όνομα του package. Για παράδειγμα:

java.awt.Color

javax.swing.JButton

- **import** Οι εντολές **import** στην αρχή του αρχείου της κλάσης επιτρέπουν την χρήση των κλάσεων βιβλιοθηκών με την σύντομη γραφή του ονόματος:

import javax.swing.JButton;

...

JButton go = new JButton("Go");

Πλήρες
(fully-qualified
) όνομα

import (συνέχεια)

- Με την χρήση του wildcard `.*` εισάγουμε όλες τις κλάσεις ενός `package`:

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;
```

Εισάγει όλες τις public κλάσεις από τα packages **awt**, **awt.event**, και **swing**

- Το `package java.lang` εισάγεται αυτόματα σε όλα τα προγράμματα. Ορίζει τις κλάσεις `System`, `Math`, `Object`, `String`, και άλλες κοινά χρησιμοποιούμενες κλάσεις.

package

- Με την εντολή `package` δηλώνουμε ότι η κλάση είναι τμήμα αυτού του `package`

Βασική δομή ορισμού μιας κλάσης

`package somerpackage;`

πακέτο στο οποίο ανήκει

`import ...`

εντολές import

`public class SomeClass`

επικεφαλίδα κλάσης

{

- Πεδία

Μεταβλητές που ορίζουν την κατάσταση (μνήμη) ενός αντικειμένου. Μπορεί να αποθηκεύουν αριθμούς, χαρακτήρες, strings, και άλλα αντικείμενα.

- Constructors

Διαδικασίες δημιουργίας νέων αντικειμένων της κλάσης και αρχικοποίησης των πεδίων τους

- Μέθοδοι

Οι ενέργειες που εκτελούν τα αντικείμενα (συμπεριφορά)

}

public class MovingObject

```
{  
    private Image picture;  
    private CoordinateSystem coordinates;  
  
    public MovingObject (int x, int y, Image pic)  
    {  
        picture = pic;  
        coordinates = new CoordinateSystem (x, y, pic);  
    }  
  
    public void moveForward (int distance)  
    {  
        coordinates.shift (distance, 0);  
    }  
  
    public void moveSideways (int distance)  
    {  
        coordinates.shift (0, distance);  
    }  
    ...  
}
```

Πεδία

Constructor

Μέθοδοι

Κλάσεις Αντικειμένων (5)

- Η πρώτη διαδικασία είναι ο constructor. Δεν έχει τύπο επιστροφής και είναι η διαδικασία που επιφορτίζεται με τη δημιουργία του αντικειμένου.
- Για να δημιουργηθεί το αντικείμενο καλείται ο constructor του, ο οποίος αρχικοποιεί ένα αντικείμενο και δημιουργεί όπως λέμε ένα στιγμιότυπο (instance).
- Οποιοδήποτε αντικείμενο πρέπει να δημιουργηθεί.
- Για τη δημιουργία ενός object σε κάθε κλάση υπάρχει ένας constructor, ο οποίος καλείται προκειμένου να δεσμευτεί μνήμη για το object.
- Ο constructor επιφορτίζεται επίσης την αρχικοποίηση των πεδίων του αντικειμένου.

Κλάσεις Αντικειμένων (6)

- Αν σε μια κλάση δεν υπάρχει κανένας constructor, η Java θεωρεί ότι υπάρχει ένας χωρίς παραμέτρους (arguments) και με κενό σώμα
- Ο constructor δεν επιστρέφει ποτέ κάποια τιμή, απλώς δημιουργεί το αντικείμενο
- Μπορούμε να έχουμε constructors με διαφορετικά arguments όπως και μεθόδους με το ίδιο όνομα και διαφορετικά arguments. Η δυνατότητα αυτή λέγεται method overloading

Κλάσεις Αντικειμένων (7)

- Όταν δημιουργούμε μία νέα κλάση στη Java, μπορούμε να ορίσουμε την ορατότητά της, δηλαδή ποιος θα μπορεί να την χρησιμοποιήσει.
- Μια κλάση μπορεί να έχει οριστεί με το πρόθεμα `public`. Σε αυτήν την περίπτωση το όνομα του αντίστοιχου αρχείου πρέπει να είναι όμοιο με το όνομα της κλάσης (π.χ. `Light.java`). Η κλάση είναι τότε προσβάσιμη από οποιοδήποτε `package` (`public access`)
- Αν δε χρησιμοποιηθεί το πρόθεμα `public`, η κλάση είναι προσβάσιμη μόνο μέσα από το ίδιο `package` (`package access`)
- Οι τύποι των δεδομένων των πεδίων της κλάσης μπορούν να είναι

```
public class A {  
    // ορατή παντού  
}
```

```
class B {  
    // ορατή στο πακέτο  
}
```

Κλάσεις Αντικειμένων (8)

- Εκτός από τον προσδιοριστή `public`, υπάρχουν ακόμη
- Ο `abstract` μετατρέπει μία κλάση σε αφηρημένη (δε μπορεί να δημιουργήσει αντικείμενα), ενώ ο `final` μαρκάρει μια κλάση ως τελική (δεν μπορεί να επεκταθεί μέσω κληρονομικότητας).

```
public abstract class A {  
    // αφηρημένη κλάση  
}
```

```
public final class B {  
    // τελική κλάση  
}
```


Προθέματα μελών κλάσεων

- Μποροστά από τα μέλη της κλάσης στην γλώσσα Java μπορούν να τοποθετηθούν ορισμένα ειδικά προθέματα που δηλώνουν την πρόσβαση που επιτρέπεται στα μέλη
- Προθέματα:
 - `public`, το μέλος είναι προσβάσιμο από οπουδήποτε αλλού
 - `private`, το μέλος είναι προσβάσιμο μόνο εντός της ίδιας της κλάσης
 - `protected`, το μέλος είναι προσβάσιμο μόνο από υποκλάσεις της κλάσης και εντός του ίδιου `package`
 - όταν δεν μπαίνει κάποιο πρόθεμα το μέλος είναι προσβάσιμο μόνον εντός του ίδιου `package` κλάσεων, που είναι μια βιβλιοθήκη από κλάσεις.

Primitive Types στην Java (1)

- Δεν είναι objects, αλλά μεταβλητές όπως στις μη-αντικειμενοστραφείς γλώσσες προγραμματισμού.
 - boolean (1 bit)
 - char (16 bits)
 - byte (8 bits)
 - int (32 bits)
 - double (64 bits)
 - short (16 bits)
 - long (64 bits)
 - float

Primitive Types στην Java (2)

- Για κάθε primitive type υπάρχει ένα αντίστοιχο class type, στην περίπτωση που θέλουμε να αποθηκεύσουμε στο heap.
 - Π.χ. `char c = 'x';`
`Character C = new Character(c);`

Δεσμευμένες λέξεις (reserved words)

- Στη java ενός αριθμός λέξεων έχουν δεσμευτεί για ειδικό σκοπό.
- Οι δεσμευμένες λέξεις χρησιμοποιούν μόνο μικρά γράμματα.
- Οι δεσμευμένες λέξεις περιλαμβάνουν:
 - βασικούς τύπους δεδομένων: int, double, char, boolean, κλπ.
 - modifiers: public, private, static, final, κλπ.
 - Εντολές ελέγχου: if, else, switch, while, for, κλπ.
 - built-in σταθερές: true, false, null
- Υπάρχουν συνολικά περίπου 50 δεσμευμένες λέξεις.

Λέξεις ορισμένες από τον προγραμματιστή

- Επιπλέον, η java χρησιμοποιεί στάνταρ ονόματα για packages και κλάσεις βιβλιοθηκών:
String, Graphics, JFrame, JButton,
java.awt, javax.swing
- Ο προγραμματιστής πρέπει να δώσει ονόματα στις δικές του κλάσεις, μεθόδους, πεδία και τοπικές μεταβλητές.

Ονόματα (συνέχεια)

- Σύνταξη: Ένα όνομα μπορεί να αποτελείται:
 - από μικρά και κεφαλαία γράμματα
 - από ψηφία
 - από τον χαρακτήρα underscore
- Σύνταξη: Ένα όνομα δεν μπορεί να αρχίζει από ψηφίο.
- Στυλ: Τα ονόματα πρέπει να είναι αρκετά περιγραφικά ώστε να διευκολύνουν την ανάγνωση ενός προγράμματος (readability).

Ονόματα (συνέχεια)

- Οι java προγραμματιστές ακολουθούν συγκεκριμένους κανόνες για τα ονόματα.
- Στυλ: τα ονόματα των κλάσεων πρέπει να αρχίζουν με κεφαλαίο γράμμα και συνηθίζεται αυτό και στις επόμενες λέξεις:
`public class DanceFloor`
- Στυλ: τα ονόματα των μεθόδων, πεδίων και τοπικών μεταβλητών αρχίζουν με μικρό γράμμα ενώ οι ακόλουθες λέξεις με κεφαλαίο:
`private int stepsCount = 0;
public void nextStep()`

Blocks, Εσοχές (συνέχεια)

- Τα άγκιστρα '{','}' οριοθετούν εμφωλιασμένα (nested) blocks.
- Τα άγκιστρα δηλώνουν ότι οι εντολές μέσα σε αυτά αποτελούν μία σύνθετη εντολή.
- Για τις εντολές μέσα σε ένα block χρησιμοποιούνται συνήθως εσοχές 2 ή 4 κενών ή ένας χαρακτήρας tab.


```
public void makeNextStep ( )
```

Το σώμα της
μεθόδου

```
{
```

```
    if (currentState == State.READY)
```

```
    {
```

```
        amy.firstStep();  
        amy.firstStep();  
        ben.firstStep();  
        currentState = State.MOVING;  
        stepsCount = 0;  
    }
```

Σύνθετη
εντολή

```
    else if (currentState == State.MOVING)
```

```
    {
```

```
        if (amy.distanceTraveled() <= danceFloor.getWidth() * 3 / 4)
```

```
        {
```

```
            amy.nextStep();  
            ben.nextStep();  
            stepsCount++;  
        }
```

Σύνθετη
εντολή

```
        else
```

```
        {
```

```
            amy.stop();  
            ben.stop();  
            currentState = State.STOPPED;  
        }
```

Σύνθετη
εντολή

```
    }
```

```
    danceFloor.update(this);
```

```
}
```

Σύνθετη
εντολή

Μεταβλητές

- Υπάρχουν 3 ειδών μεταβλητές
 - πεδία
 - τοπικές μεταβλητές
 - παράμετροι
- Μία μεταβλητή πρέπει να δηλωθεί πριν χρησιμοποιηθεί:

The diagram shows a list of variable declarations on a light blue background. Each declaration has its type and variable name highlighted with a red box. Annotations include a yellow box labeled 'Τύπος' (Type) with an arrow pointing to the 'double' type in the second line, and another yellow box labeled 'Ονόματα' (Names) with an arrow pointing to the 'count' variable in the first line.

```
int count;  
double x, y;  
JButton go;  
Walker amy;  
String firstName;
```

Μεταβλητές (συνέχεια)

- Ο τελεστής ανάθεσης (assignment operator) = θέτει την τιμή μιας μεταβλητής:

```
count = 5;  
x = 0;  
go = new JButton("Go");  
firstName = args[0];
```

Μεταβλητές (συνέχεια)

- Μία μεταβλητή μπορεί να αρχικοποιηθεί στην δήλωσή της:

```
int count = 5;  
JButton go = new JButton("Go");  
String firstName = args[0];
```

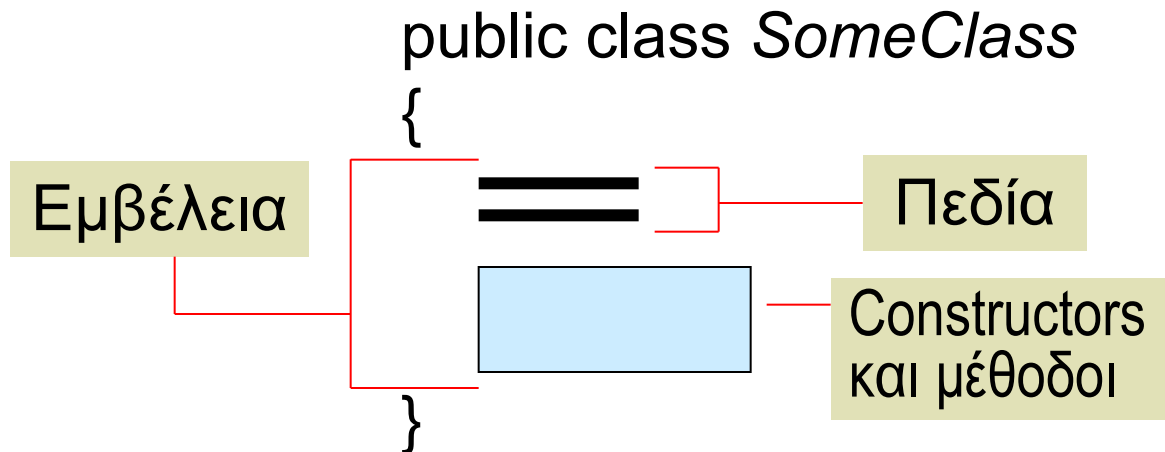
Μεταβλητές: Εμβέλεια

- Κάθε μεταβλητή έχει την δική της εμβέλεια
- Η χρήση μιας μεταβλητής εκτός της εμβέλειάς της είναι συντακτικό λάθος.
- Οι μεταβλητές των οποίων οι εμβέλειες δεν επικαλύπτονται μπορούν να έχουν το ίδιο όνομα.

```
{  
    int k = ...;  
    ...  
}  
  
for (int k = ...)  
{  
    ...  
}
```

Πεδία (Fields ή Member variables)

- Τα πεδία δηλώνονται εκτός των ορίων των constructor και των μεθόδων.
- Η εμφάνιση των πεδίων είναι όλη η κλάση.



Τοπικές μεταβλητές

- Οι τοπικές μεταβλητές δηλώνονται μέσα σε έναν constructor ή σε μία μέθοδο.
- Οι τοπικές μεταβλητές καταστρέφονται κατά την έξοδο από τον constructor ή την μέθοδο.
- Η εμβέλεια μιας τοπικής μεταβλητής εκτείνεται από το σημείο της δήλωσής της έως το δεξί άγκιστρο του block μέσα στο οποίο δηλώνεται.

Τοπικές μεταβλητές (συνέχεια)

```
public class SomeClass  
{
```

```
    ...
```

```
    public SomeType SomeMethod (...)
```

```
    {
```

```
        _____
```

Δήλωση τοπικής μεταβλητής

```
        _____  
        _____
```

```
        {
```

```
            _____  
            _____
```

Δήλωση τοπικής μεταβλητής

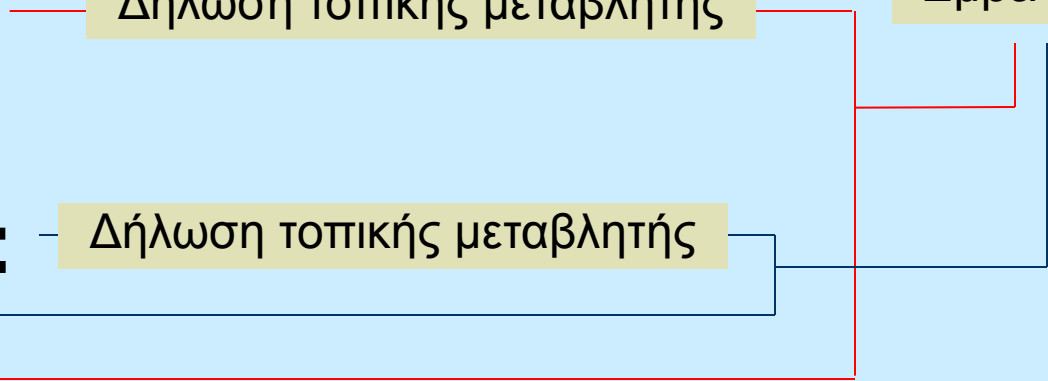
```
        }
```

```
    }
```

```
    ...
```

```
}
```

Εμβέλεια



Strings

- Ο τύπος `String` δεν ανήκει στους βασικούς τύπους δεδομένων
- Τα `String` συμπεριφέρονται όπως όλα τα άλλα αντικείμενα αλλά με δύο εξαιρέσεις:
 - Δεν χρειάζεται η κλήση του constructor για τη δημιουργία τους.
 - Τα `String` μέσα σε διπλά εισαγωγικά (double quotes) είναι σταθερές
 - Οι τελεστές `+` και `+=` συνενώνουν strings (ή ένα string και έναν αριθμό ή αντικείμενο τα οποία μετατρέπονται πρώτα σε string)
 - Είναι `immutable objects`. Αυτό σημαίνει ότι αφού δημιουργηθούν τα δεδομένα τους στη μνήμη δεν μπορούν να αλλάξουν.

"Catch " + 22



"Catch 22"

Σταθερές (Literal Constants)

new line

tab

'B', '+', '\n', '\t'

char

-85, 2012, 0

int

0.45, -11.3, 8., .4

double

"image.txt", "1756", "y", "\n"

String

Access Modifiers

- `public`: ορατά από όλες τις κλάσεις
- `private`: ορατά από την παρούσα κλάση μόνο
- `protected`: ορατά από τις κλάσεις του `package` και από όλες τις subclasses
- `no modifier`: ορατά από το `package` (`package-private`)
- στο επίπεδο της κλάσης έχουν νόημα οι modifiers `public` και ο `default` (`package-private`).

Public ή Private

- Οι constructors και μέθοδοι που δηλώνονται public αποτελούν το interface μιας κλάσης.
- Τα πεδία μιας κλάσης δηλώνονται συνήθως private ώστε να μην είναι ορατά από τις άλλες κλάσεις.
- Οι συμβολικές σταθερές (constants) μπορεί να είναι public.
- Οι “helper” μέθοδοι χρησιμοποιούνται μόνο μέσα σε μία κλάση και δηλώνονται private.

Public ή Private ή ...

- Η Java υποστηρίζει τέσσερα διαφορετικά επίπεδα ορατότητας, το `public`, το `protected`, το `default` και το `private`.

Ορατότητα	<code>public</code>	<code>protected</code>	<code>default</code>	<code>private</code>
Από την ίδια κλάση	Ναι	Ναι	Ναι	Ναι
Από άλλη κλάση στο ίδιο πακέτο	Ναι	Ναι	Ναι	Όχι
Από οποιαδήποτε μη υποκλάση σε άλλο πακέτο	Ναι	Όχι	Όχι	Όχι
Από υποκλάση στο ίδιο πακέτο	Ναι	Ναι	Ναι	Όχι
Από υποκλάση σε άλλο πακέτο	Ναι	Ναι	Όχι	Όχι

Public ή Private (συνέχεια)

- Ένα `private` πεδίο είναι προσπελάσιμο οπουδήποτε μέσα στην κλάση.
- Κάθε αντικείμενο μπορεί να προσπελάσει και να τροποποιήσει οποιοδήποτε αντικείμενο της ίδιας κλάσης.

```
public class Fraction
{
    private int num, denom;
    ...
    public multiply (Fraction other)
    {
        int newNum = num * other.num;
        ...
    }
}
```

Accessors και Modifiers μέθοδοι

- Οι μέθοδοι οι οποίες επιστρέφουν την τιμή ενός `private` πεδίου καλούνται *accessors*. Οι μέθοδοι που δίνουν τιμές σε `private` πεδία καλούνται *modifiers*.
- Τα ονόματα των `accessor` μεθόδων συνήθως αρχίζουν από `get` και των `modifier` μεθόδων από `set`.

Ενθυλάκωση (encapsulation)

- Η απόκρυψη των λεπτομερειών υλοποίησης μίας κλάσης (private πεδία και helper μέθοδοι) καλείται ενθυλάκωση (*encapsulation*).
- Η ενθυλάκωση βοηθά στην συντήρηση του κώδικα: μια εσωτερική αλλαγή σε μία κλάση δεν επηρεάζει τις άλλες κλάσεις.
- Μία κλάση αλληλεπιδρά με μία άλλη κλάση μέσω καλά ελεγμένων και τεκμηριωμένων constructor και μεθόδων.

Οι constructors

- Ένας constructor είναι μία διαδικασία η οποία δημιουργεί αντικείμενα μιας κλάσης.
- Συχνά ένας constructor αρχικοποιεί τα πεδία ενός αντικειμένου.
- Οι constructors δεν έχουν τύπο επιστροφής (ούτε καν `void`) και συνεπώς δεν επιστρέφουν καμία τιμή.
- Όλοι οι constructors μιας κλάσης έχουν το ίδιο όνομα (το όνομα της κλάσης).
- Οι constructors μπορούν να δέχονται παραμέτρους.

Οι constructors (συνέχεια)

- Εάν μία κλάση έχει περισσότερους constructors αυτοί πρέπει να διαφέρουν στον αριθμό και τύπο των παραμέτρων.
- Μπορεί να υπάρχει constructor χωρίς παραμέτρους (no-args).
- Εάν δεν ορίζονται constructors η Java δημιουργεί αυτόματα έναν no-arg constructor ο οποίος δεσμεύει μνήμη για τα αντικείμενα και δίνει default τιμές στα πεδία.
- Προσοχή! Ο default constructor καλεί τον no-arg constructor της superclass, ο οποίος πρέπει να είναι ορισμένος.

Οι constructors (συνέχεια)

```
public class Fraction
{
    private int num, denom;
```

```
    public Fraction ( )
```

```
    {
        num = 0;
        denom = 1;
    }
```

“No-arg”
constructor

```
    public Fraction (int n)
```

```
    {
        num = n;
        denom = 1;
    }
```

Continued □

```
    public Fraction (int n, int d)
```

```
    {
        num = n;
        denom = d;
        reduce ();
    }
```

```
    public Fraction (Fraction other)
```

```
    {
        num = other.num;
        denom = other.denom;
    }
    ...
```

Copy
constructor

```
}
```

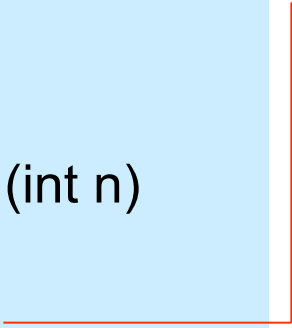
Οι constructors (συνέχεια)

- Οι constructors μιας κλάσης μπορούν να καλούν ο ένας τον άλλον με χρήση της δεσμευμένης λέξης **this**:

```
public class Fraction
{
    ...

    public Fraction (int n)
    {
        this (n, 1);
    }
    ...
}
```

```
...
public Fraction (int p, int q)
{
    num = p;
    denom = q;
    reduce ();
}
...
```



Δημιουργία Αντικειμένων

- Για να δημιουργήσουμε ένα αντικείμενο κάποιας κλάσης, χρειαζόμαστε πρωτίστως μία αναφορά ιδίου τύπου με αυτόν της κλάσης από την οποία θέλουμε να δημιουργήσουμε αντικείμενο. Έτσι λοιπόν, αν η κλάση μας λεγόταν MyClass θα γράφαμε:

MyClass ref = new MyClass();

δηλώνεται μία αναφορά
τύπου MyClass με το
όνομα ref.

ο compiler καλεί έμμεσα
τον default constructor της
MyClass, οπότε δεσμεύεται
η κατάλληλη μνήμη και
δημιουργείται το
αντικείμενο.

Ο τελεστής new

- Οι constructors καλούνται μέσω του τελεστή new.
- Οι παράμετροι που περνιούνται στον new πρέπει να ταιριάζουν στον αριθμό, την σειρά και τον τύπο με έναν constructor.

```
Fraction f1 = new Fraction ( );  
Fraction f2 = new Fraction (5);  
Fraction f3 = new Fraction (4, 6);  
Fraction f4 = new Fraction (f3);
```

```
public class Fraction  
{  
    public Fraction (int n)  
    {  
        num = n;  
        denom = 1;  
    }  
    ...  
}
```

5 / 1

Ο τελεστής `new` (συνέχεια)

- Με τον τελεστή `new` δημιουργείται ένα καινούργιο αντικείμενο (class instantiation).
- Οι παράμετροι χρησιμοποιούνται για την αρχικοποίηση του αντικειμένου (object initialization).

```
private Fraction ratio;
```

Η **ratio** είναι **null**

```
...
```

```
ratio = new Fraction (2, 3);
```

Εδώ η **ratio** αναφέρεται σε ένα αντικείμενο

```
...
```

```
ratio = new Fraction (3, 4);
```

Εδώ η **ratio** αναφέρεται σε ένα άλλο αντικείμενο (το προηγούμενο αντικείμενο καταστρέφεται ως αποτέλεσμα του “garbage collection”)

Πεδία (Fields)

- Επίσης γνωστά ως *instance* ή *member* μεταβλητές
- Αποτελούν την μνήμη (ιδιωτική) ενός αντικειμένου
- Κάθε πεδίο χαρακτηρίζεται από έναν τύπο δεδομένων (int, double, String, Image, MovingObject, etc.)
- Κάθε πεδίο έχει ένα όνομα που δίνεται από τον προγραμματιστή

Μέθοδοι: Το στυλ της Java

- Το όνομα μιας μεθόδου αρχίζει με μικρό γράμμα.
- Το όνομα της μεθόδου που επιστρέφει την τιμή ενός πεδίου συχνά αρχίζει με `get` (accessor)
`getWidth`, `getX`
- Το όνομα της μεθόδου που δίνει τιμή σε ένα πεδίο συχνά αρχίζει με `set` (modifier)
`setLocation`, `setText`

Overloaded μέθοδοι

- Μέθοδοι της ίδιας κλάσης που έχουν το ίδιο όνομα αλλά διαφορετικό αριθμό ή τύπο παραμέτρων καλούνται *overloaded*.
- Χρησιμοποιούνται όταν εκτελούν όμοιες λειτουργίες:

```
public void move (int x, int y) { ... }  
public void move (double x, double y) { ... }  
public void move (Point p) { ... }
```

```
public Fraction add (int n) { ... }  
public Fraction add (Fraction other) { ... }
```

Overloaded μέθοδοι (συνέχεια)

- Ο compiler μεταχειρίζεται τις overloaded μεθόδους ως τελείως διαφορετικές μεθόδους.
- Ο compiler γνωρίζει ποια να καλέσει κάθε φορά με βάση τον αριθμό και τον τύπο των ενεργών παραμέτρων.

```
Circle circle = new Circle(5);  
circle.move (50, 100);  
Point center =  
    new Point(50, 100);  
circle.move (center);
```

```
public class Circle  
{  
    public void move (int x, int y)  
        { ... }  
  
    public void move (Point p)  
        { ... }  
    ...  
}
```

Overloaded μέθοδοι (συνέχεια)

- Ο επιστρεφόμενος τύπος δεν αρκεί για να διακρίνει ο compiler ποια από τις μεθόδους καλείται.

```
public class Circle
{
    public void move (int x, int y)
    { ... }

    public Point move (int x, int y)
    { ... }
    ...
}
```

Συντακτικό
λάθος

Static πεδία

- Η τιμή ενός *static* πεδίου [καλείται και μεταβλητή κλάσης (class variable)] μιας κλάσης είναι κοινή σε όλα τα αντικείμενα της κλάσης.
- Ένα non-static πεδίο [καλείται και μεταβλητή στιγμιότυπου (instance variable)] ανήκει μόνο σε ένα αντικείμενο.

Static πεδία (συνέχεια)

- Τα static πεδία αποθηκεύονται μαζί με τον κώδικα της κλάσης και μνήμη γι' αυτά δεσμεύεται μία μόνο φορά, όταν φορτώνεται η κλάση στην μνήμη.
- Τα static πεδία που είναι public (συνήθως καθολικές (global) σταθερές, αναφέρονται από τις άλλες κλάσεις ως εξής:

ClassName.constName

```
double area = Math.PI * r * r;  
setBackground(Color.BLUE);  
c.add(btn, BorderLayout.NORTH);  
System.out.println(area);
```

Πεδία (συνέχεια)

- Τα static πεδία συνήθως αρχικοποιούνται με μία δομή της γλώσσας που λέγεται static initialization block

```
public class MyInteger
{
    static int minValue;
    static int maxValue;

    static
    {
        minValue=0;
        maxValue=16535;
    }
}
```

Static πεδία (συνέχεια)

- Συνήθως τα static πεδία δεν αρχικοποιούνται μέσα σε constructors αλλά είτε κατά την δήλωσή τους είτε μέσα σε public static μεθόδους.
- Εάν μία κλάση περιέχει μόνο static πεδία και μεθόδους, δεν υπάρχει λόγος να δημιουργούνται αντικείμενα αυτής της κλάσης.
- Οι κλάσεις Math και System είναι παραδείγματα τέτοιων μεθόδων. Δεν έχουν public constructors και δεν μπορούμε να δημιουργήσουμε στιγμιότυπα (αντικείμενα) για αυτές

Static μέθοδοι

- Οι Static μέθοδοι μπορούν να προσπελάσουν μόνο static πεδία.
- Απαγορεύεται να προσπελάσουν non-static πεδία ή να καλέσουν non-static μεθόδους.
- Οι static μέθοδοι καλούνται ως εξής:
ClassName.statMethod(...)

```
double x = Math . random();  
double y = Math . sqrt (x);  
System . exit();
```

Static μέθοδοι (συνέχεια)

```
public class MyClass
{
    public static final int statConst;
    private static int statVar;

    private int instVar;
    ...
    public static int statMethod(...)
    {
        statVar = statConst;
        statMethod2(...);

        instVar = ...;
        instMethod(...);
    }
}
```

Static
μέθοδος

OK

Errors!

Η μέθοδος main()

- Κάθε Java application έχει μία κλάση η οποία περιέχει την μέθοδο main() (main class).
- Η εκτέλεση του προγράμματος αρχίζει από αυτή την μέθοδο.
- Το signature (όνομα και παράμετροι) της μεθόδου main() είναι το εξής:

```
public static void main(String[] args)
```

- Η παράμετρος **args** είναι ένα array από strings που περιέχει τις παραμέτρους του προγράμματος

Η μέθοδος main()

Public static void main (string args[])

Public : μπορεί να κληθεί από άλλη method άλλης class που έχει πρόσβαση στη class της main

Static : μπορεί να κληθεί ακόμα και αν δεν υπάρχει αντικείμενο της τάξης

Void : η method δεν επιστρέφει τιμή

Main : όνομα της method

String args[] : παράμετροι της main

Hello world program

```
public class Main {  
    /**  
     * This is the entry point into the application.  
     * The main method MUST be declared "public", "static", with "void" return  
     * type and one parameter of type "String[]".  
     * Note: The class that contains the main method can have any valid name,  
     * as long as the .java file has the same name.  
     * Here, we choose to call it "Main" just to denote that this is the class where  
     * the main method is been declared.    */  
    public static void main(String[] args){  
        // System : a Java class  
        // out      : a field of System class [of type PrintStream]  
        // println : a method of PrintStream class  
        System.out.println("Hello World of Java");  
    }  
}  
/**  
 * Class A is visible only from classes that belong to the same package  
 * In our case, this is the default-package.  
 */  
class A{/* Class' definition goes here */}
```

Static μέθοδοι (συνέχεια)

- Η `main` μέθοδος είναι `static` και επομένως δεν μπορεί να προσπελάσει `non-static` πεδία ή να καλέσει `non-static` μεθόδους της κλάσης που ανήκει:

```
public class Hello
{
    private int test () { ... }

    public static void main (String[ ] args)
    {
        System.out.println (test ());
    }
}
```

Error:

Η `non-static` μέθοδος **test** καλείται από την `static main`

Non-Static μέθοδοι

- Μία non-static μέθοδος καλείται για ένα συγκεκριμένο αντικείμενο ως εξής:

```
objName.instMethod(...);
```

```
vendor.addMoney(25);
```

```
die1.roll();
```

- Οι non-static μέθοδοι μπορούν να προσπελάσουν όλα τα πεδία μιας κλάσης και να καλέσουν όλες τις μεθόδους της, static και non-static.

Η κλάση String

- Ένα αντικείμενο της κλάσης String αντιπροσωπεύει ένα string από χαρακτήρες.
- Η κλάση String ανήκει στο package `java.lang` το οποίο είναι default package της Java.
- Όπως όλες οι άλλες κλάσεις, η String έχει constructors και μεθόδους.
- Σε αντίθεση με άλλες κλάσεις, η String έχει δύο τελεστές, τον `+` και `+=` για συνένωση (concatenation) δύο string.

Πραγματικές σταθερές String

- Οι πραγματικές (literal) σταθερές της κλάσης String αποτελούν ανώνυμα αντικείμενα-σταθερές και στην ουσία είναι συμβολοσειρές μέσα σε διπλά εισαγωγικά, π.χ. "Hello World".

- Αποτελούν τις τιμές των String μεταβλητών.
- Μπορούν να αποτελέσουν παραμέτρους των μεθόδων και constructors.
- Ως αντικείμενα, έχουν μεθόδους

```
String fileName = "fish.dat";  
button = new JButton("Next slide");  
if ("Start" . equals(cmd)) ...
```

Πραγματικές σταθερές String (συνέχεια)

Μπορούν να περιλαμβάνουν escape χαρακτήρες. Για παράδειγμα:

- \\ αντιπροσωπεύει τον χαρακτήρα \
- \n αντιπροσωπεύει τον newline χαρακτήρα

```
String s1 = "Biology";  
String s2 = "C:\\jdk1.4\\docs";  
String s3 = "Hello\n";
```

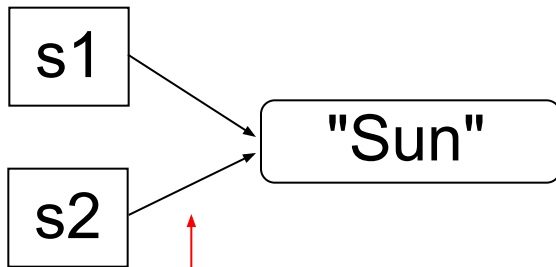
Immutability

- Ένα αντικείμενο String δημιουργείται μία φορά και παραμένει καθ' όλη την διάρκεια της ύπαρξής του αμετάβλητο.
- Τέτοια αντικείμενα ονομάζονται *immutable*.
- Για τα *immutable* αντικείμενα μπορούν να υπάρχουν περισσότερες από μία αναφορές (references) με ασφάλεια αφού δεν υπάρχει ο κίνδυνος λανθασμένης αλλαγής αυτών.

Immutability (συνέχεια)

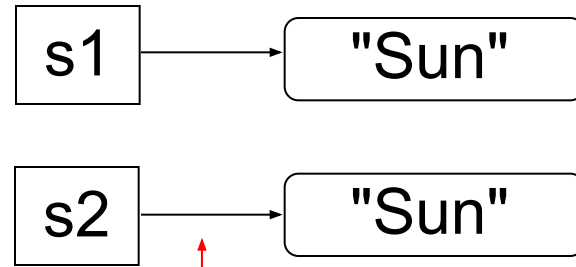
Πλεονέκτημα: η χρήση των objects είναι πιο αποτελεσματική.

```
String s1 = "Sun";  
String s2 = s1;
```



OK

```
String s1 = "Sun";  
String s2 = new String(s1);
```

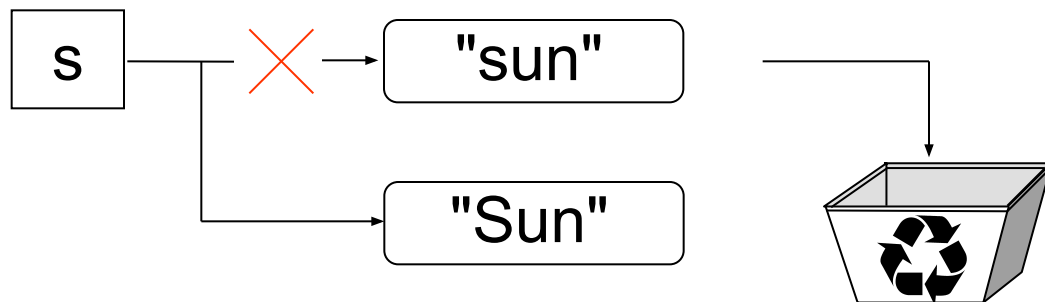


σπατάλη μνήμης

Immutability (συνέχεια)

Μειονέκτημα: δημιουργείται ένα καινούργιο string για κάθε αλλαγή, όσοδήποτε μικρή.

```
String s = "sun";  
char ch = Character.toUpperCase(s.charAt (0));  
s = ch + s.substring (1);
```



Κενά Strings

Ένα κενό string δεν έχει καθόλου χαρακτήρες και το μήκος του είναι 0.

```
String s1 = "";
```

```
String s2 = new String();
```

κενά strings



Δεν πρέπει να συγχέεται με μη αρχικοποιημένο string:

```
private String errorMsg;
```

**To errorMsg
είναι null**



String μέθοδοι — length, charAt

`int length ();`

`char charAt (k);`

Επιστρέφει τον αριθμό
χαρακτήρων του string
επιστρέφει τον k-οστό
χαρακτήρα

Επιστρέφει:

`"Flower".length();`

`"Wind".charAt (2);`

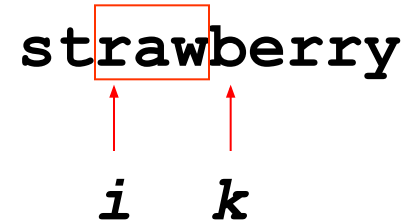
6

'n'

String μέθοδοι — substring

String s2 = s.**substring** (i, k);

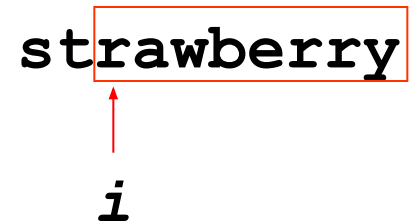
επιστρέφει το substring από την i-οστή στην (k-1)-οστή θέση



The diagram shows the string "strawberry". A red box highlights the characters "raw". Below the 'r' is a red arrow pointing up, labeled with the variable *i*. Below the 'y' is a red arrow pointing up, labeled with the variable *k*.

String s2 = s.**substring** (i);

επιστρέφει το substring από την i-οστή θέση μέχρι το τέλος



The diagram shows the string "strawberry". A red box highlights the characters "rawberry". Below the 'r' is a red arrow pointing up, labeled with the variable *i*.

"strawberry".substring (2,5);

"unhappy".substring (2);

"emptiness".substring (9);

επιστρέφει:

→ "raw"

→ "happy"

→ "" (empty string)

String μέθοδοι — Concatenation

String result = s1 + s2;

συνενώνει τα s1 και s2

String result = s1.concat (s2);

το ίδιο όπως s1 + s2

result += s3;

συνενώνει το s3 με το result

result += num;

μετατρέπει το **num** σε **String** και το συνενώνει με το result

String Μέθοδοι — Εύρεση (indexOf)

0 8 11 15

String date ="July 5, 2012 1:28:19 PM";

Επιστρέφει:

date.indexOf ('J'); 0

date.indexOf ('2'); 8

date.indexOf ("2012"); 8

date.indexOf ('2', 9); 11

(αρχίζει από την
θέση 9)

date.indexOf ("2020"); -1

date.lastIndexOf ('2'); 15

(δεν βρέθηκε)

EvenOdd

- Να δημιουργήσετε πρόγραμμα που διαβάζει έναν αριθμό από το πληκτρολόγιο και εμφανίζει στον χρήστη ένα μήνυμα αν είναι άρτιος η περιττός.

EvenOdd

```
import java.util.Scanner;
/**
 * Basic example for finding whether a number entered by the user is even or odd
 */
public class EvenOdd {

    public static void main(String[] args) {
        // We pass System.in to Scanner class' constructor
        // We create a Scanner object
        Scanner input = new Scanner(System.in);
        int num;
        System.out.println("Enter an Integer number:");
        // If the input token is not an integer, an InputMismatchException
        // is thrown by the nextInt method.
        num = input.nextInt();

        if(num % 2 == 0)
            System.out.println("Entered number is even");
        else
            System.out.println("Entered number is odd");
    }
    input.close();
}
```

Arrays

Ένα array είναι ένα τμήμα διαδοχικών θέσεων μνήμης κάθε μία από τις οποίες έχει δεδομένα ίδιου τύπου.

Στην Java, το k -οστό στοιχείο του array a προσπελάζεται ως $a[k]$, όπου k είναι ο δείκτης. Το πρώτο στοιχείο του array a είναι το $a[0]$ ενώ το τελευταίο είναι το $a[n-1]$ όπου n το μήκος του array.

Στη Java, ένα array δηλώνεται με συγκεκριμένο μήκος το οποίο δεν αλλάζει κατά την διάρκεια εκτέλεσης.

Arrays (συνέχεια)

Παραδείγματα αναφορών σε στοιχεία ενός array

```
a [3]
```

```
a [k]
```

```
a [k - 2]
```

```
a [ (int) (6 * Math.random()) ]
```

Arrays ως αντικείμενα

Στη Java, ένα array είναι ουσιαστικά ένα αντικείμενο. Εάν ο τύπος των στοιχείων του είναι *anyType*, ο τύπος του array αντικειμένου είναι *anyType[]*.

Δήλωση Array:

```
anyType [ ] arrName;
```

Όπως και με τα άλλα αντικείμενα, η δήλωση ενός array δημιουργεί μία αναφορά και όχι το ίδιο το αντικείμενο.

Τρόπος δημιουργίας ενός array:

```
arrName = new anyType [length] ;
```

αγκύλες, όχι
παρενθέσεις!



Δήλωση και αρχικοποίηση array

Όταν δημιουργείται ένα array, δεσμεύεται χώρος για τα στοιχεία του. Τα στοιχεία παίρνουν default τιμές. Για παράδειγμα:

```
scores = new int [10] ;
```

μήκος 10, όλα τα
στοιχεία παίρνουν τιμή 0

```
words = new String [10000];
```

μήκος 10000,
όλα τα στοιχεία
έχουν τιμή null

Δήλωση και αρχικοποίηση array (συνέχεια)

Ένα array μπορεί να δηλωθεί και να αρχικοποιηθεί με μία εντολή. Για παράδειγμα:

```
int [ ] scores = new int [10] ;
```

```
private double [ ] gasPrices = { 3.05, 3.17, 3.59 };
```

```
String [ ] words = new String [10000];
```

```
String [ ] cities = {"Atlanta", "Boston", "Cincinnati" };
```

Δήλωση και αρχικοποίηση array (συνέχεια)

Επίσης, η αρχικοποίηση μπορεί να αναβληθεί για αργότερα. Για παράδειγμα:

```
String [ ] words;
```

Δεν έχει
αρχικοποιηθεί

```
...
```

```
words = new String [ console.readInt() ];
```

```
private double[ ] gasPrices;
```

Δεν έχει
αρχικοποιηθεί

```
...
```

```
gasPrices = new double[ ] { 3.05, 3.17, 3.59 };
```

Το μήκος ενός array

- Το μήκος ενός array καθορίζεται κατά την δημιουργία του array.
- Το μήκος ενός array είτε δίνεται άμεσα, είτε υπολογίζεται από το μήκος της λίστας αρχικοποίησης.
- Το μήκος του array `arrName` προσπελαύνεται ως `arrName.length`.
- Το `length` είναι ένα `public` πεδίο και όχι μία μέθοδος του array αντικειμένου.

Περνώντας arrays ως παραμέτρους

Όπως όλα τα αντικείμενα, ένα array περνιέται σε μία μέθοδο με τη δημιουργία αντιγράφου της αναφοράς

Τα στοιχεία του αρχικού array δεν αντιγράφονται και είναι προσπελάσιμα (μεταβάλλονται) από τον κώδικα της μεθόδου.

```
// Swaps a [ i ] and a [ j ]  
public void swap (int [ ] a, int i, int j)  
{  
    int temp = a [ i ];  
    a [ i ] = a [ j ];  
    a [ j ] = temp;  
}
```

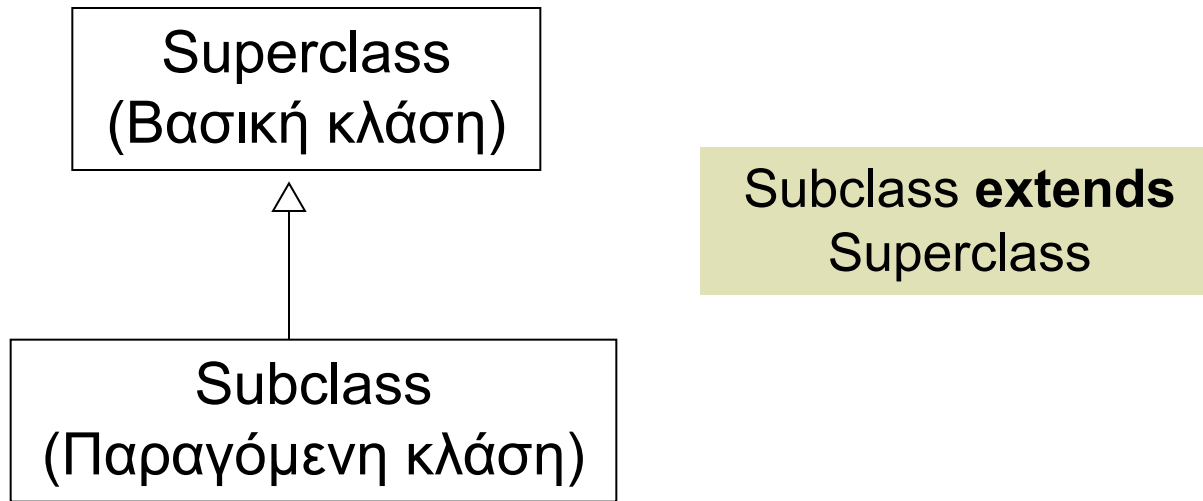
EvenOdd

- Επεκτείνουμε το πρόγραμμα ώστε να διαβάζει η αριθμούς από το πληκτρολόγιο και να τους αποθηκεύει σε πίνακες (array) άρτιους ή περιττούς αντίστοιχα.
- Τέλος να εκτυπώνει τα δύο arrays

Κληρονομικότητα (Inheritance)

- Στην Java ισχύει ένας πολύ **βασικός κανόνας** σύμφωνα με τον οποίο η **οποιαδήποτε κλάση, θα πρέπει να κληρονομεί οπωσδήποτε από κάποια άλλη.**
- Αυτό που συμβαίνει είναι πως όταν δημιουργούμε μία νέα κλάση, στην ουσία έχουμε δύο επιλογές. Η μία είναι να κληρονομήσουμε άμεσα από μία βασική κλάση.
- Η δεύτερη επιλογή που συμβαίνει στις περισσότερες περιπτώσεις, είναι αν η κλάση που δημιουργούμε δεν κληρονομεί άμεσα από κάποια άλλη, ο compiler θα την θέσει αυτόματα να κληρονομήσει από την κλάση **Object**. Αυτό συμβαίνει πάντοτε όταν μία κλάση δεν κληρονομεί άμεσα από κάποια άλλη, ακόμη κι αν δεν το βλέπετε γραμμένο ρητά στον κώδικα (δεν θα δείτε ποτέ π.χ. statement όπως `public class Car extends Object`).
- Η κλάση `Object`, είναι στην ουσία η υπερκλάση όλων των κλάσεων που υπάρχουν ή που μπορεί να δημιουργηθούν στη Java.

Κληρονομικότητα (Inheritance)



- Η κληρονομικότητα καθορίζει μία IS-A σχέση μεταξύ των αντικειμένων: το αντικείμενο μιας subclass IS-A αντικείμενο της αντίστοιχης superclass.

Κληρονομικότητα (2 – subclass)

- Έχει πρόσβαση στα πεδία και τις μεθόδους (public, protected, ή package-private όταν βρίσκεται στο ίδιο package) της υπερκείμενης κλάσης (superclass ή parent class)
- μπορεί να προσθέτει νέα πεδία και μεθόδους
- μπορεί να ορίζει ξανά (override) μία μέθοδο της superclass εκτός αν έχει δηλωθεί ως final
- πρέπει να έχει τους δικούς της constructors οι οποίοι μπορούν να καλούν τους constructors της superclass
- δεν μπορεί να έχει άμεση πρόσβαση στα private πεδία της superclass
- Προσοχή! Μία κλάση η οποία έχει δηλωθεί ως final δεν μπορεί να έχει subclasses.

Κληρονομικότητα (3)

- Είναι όμως δυνατόν να οριστεί εκ νέου μία κληρονομούμενη μέθοδος (**overriding**) της μητέρας κλάσης. Αυτό δεν μπορεί να γίνει για μέλη τα οποία έχουν οριστεί ως final.
- Κανόνες
 - η υπερκλάση όσο και η υποκλάση πρέπει να έχουν το ίδιο όνομα μεθόδου, τον ίδιο τύπο επιστροφής και την ίδια λίστα παραμέτρων.
 - Δεν μπορούμε να παρακάμψουμε μια μέθοδο που έχει δηλωθεί ως final ή static.
 - Πρέπει πάντα να παρακάμπτουμε (override) τις αφορημένες

```
class Animal {  
    public void displayInfo() {  
        System.out.println("I am an animal.");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    public void displayInfo() {  
        System.out.println("I am a dog.");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo();  
    }  
}
```

Output:

I am a dog.

Notice the use of the **@Override** annotation in our example. In Java, annotations are the metadata that we used to provide information to the compiler. Here, the **@Override** annotation tells the compiler that the method after this annotation overrides the method of the superclass.

Κληρονομικότητα (4)

- Μπορούμε να έχουμε πρόσβαση στη μέθοδο της υπερκλάσης μετά το overriding?
- Το keyword **super** χρησιμοποιείται στα παιδιά για να κληθεί η μέθοδος της μητέρας κλάσης.

In the example, the subclass Dog overrides the method displayInfo() of the superclass Animal.

*When we call the method displayInfo() using the d1 object of the Dog subclass, **the method inside the Dog subclass is called; the method inside the superclass is not called.***

Inside displayInfo() of the Dog subclass, we have used super.displayInfo() to call displayInfo() of the superclass.

```
class Animal {  
    public void displayInfo() {  
        System.out.println("I am an animal.");  
    }  
}  
  
class Dog extends Animal {  
    public void displayInfo() {  
        super.displayInfo();  
        System.out.println("I am a dog.");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo();  
    }  
}
```

Output:

```
I am an animal.  
I am a dog.
```

Κληρονομικότητα(5)

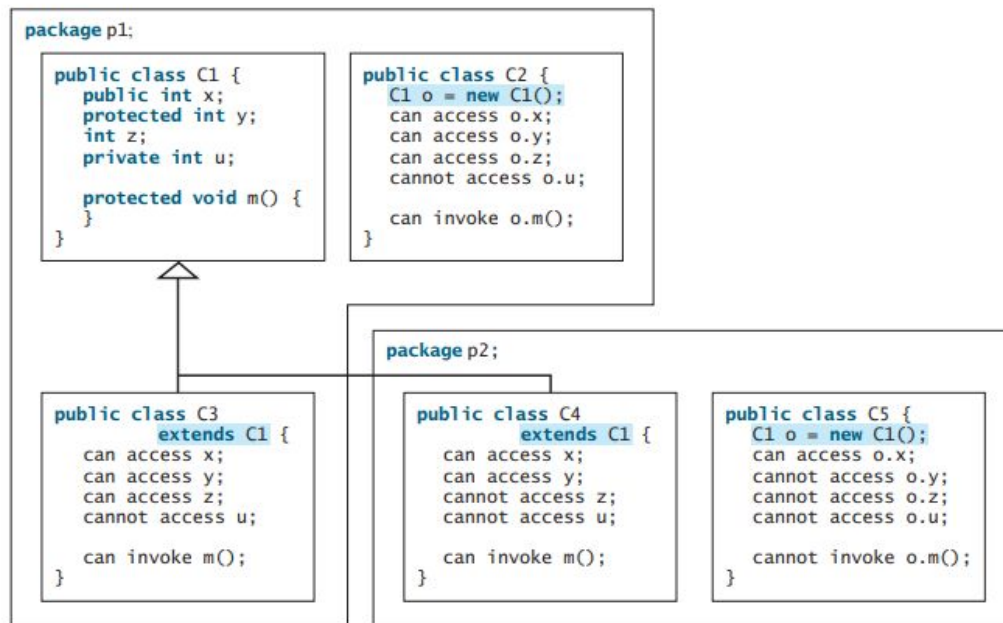
- Στην περίπτωση που ο constructor της μητέρας-κλάσης δέχεται arguments θα πρέπει η πρώτη κλήση του constructor της κλάσης-παιδιού να καλεί τον constructor της μητέρας-κλάσης με τα απαραίτητα arguments.
- Αν δεν συμβαίνει αυτό, τότε καλείται implicitly ο constructor χωρίς arguments (είτε γραφεί στον κώδικα είτε όχι). Αν η μητέρα-κλάση δεν έχει constructor χωρίς arguments, θα δημιουργηθεί σφάλμα μεταγλώττισης.
- Μια άλλη σημαντική ιδιότητα είναι το *upcasting*: Αν μία μέθοδος δέχεται ως argument τη μητέρα-κλάση, τότε θα δέχεται και οποιαδήποτε κλάση την κληρονομεί.

Κληρονομικότητα(6)

- If a class contains an abstract method, it is **mandatory** to override it. .. Θα το αναλύσουμε και στην συνέχεια αυτό

Κληρονομικότητα (7)

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass in a different package	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	-
default (no modifier)	✓	✓	-	-
private	✓	-	-	-



Κληρονομικότητα(8)

- Η ίδια μέθοδος που δηλώνεται στην υπερκλάση και τις υποκλάσεις της μπορεί να έχει ορίσει διαφορετικούς τρόπους πρόσβασης. Ωστόσο, υπάρχει ένας περιορισμός.
- We can only use those access specifiers in subclasses that provide larger access than the access specifier of the superclass.
- Για παράδειγμα: Ας υποθέσουμε ότι μια μέθοδος `myClass()` στην υπερκλάση δηλώνεται ως `protected`. Στην υποκλάση, η ίδια μέθοδος `myClass()` μπορεί να είναι είτε `public` είτε `protected`, αλλά **όχι `private`**.

```
public class Car extends MovingObject
```

```
{
```

```
    public Car (int x, int y, Image pic)
```

```
    {
```

```
        super(x, y, pic);
```

```
    }
```



Constructor

Καλεί τον constructor της **MovingObject** μέσω του **super** constructor

```
public void turnAround ()
```

```
{
```

```
    this.moveSideways(-10);
```

```
    this.moveSideways(-10);
```

```
    this.moveSideways(-10);
```

```
    this.moveSideways(10);
```

```
    this.moveForward(10);
```

```
}
```

Νέα
μέθοδος

Συνέχεια ορισμού της κλάσης Car

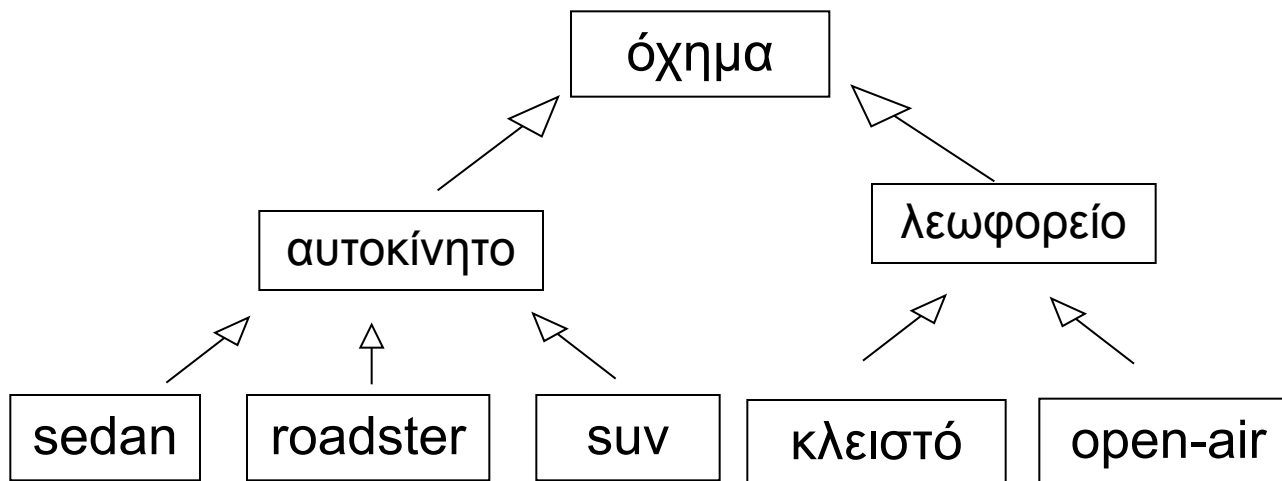
```
...  
public void moveSideways(int distance)  
{  
    super.moveSideways(-distance);  
}  
...  
}
```

Override της
μεθόδου
moveSideways της
superclass
MovingObject

καλεί την μέθοδο moveSideways
της superclass

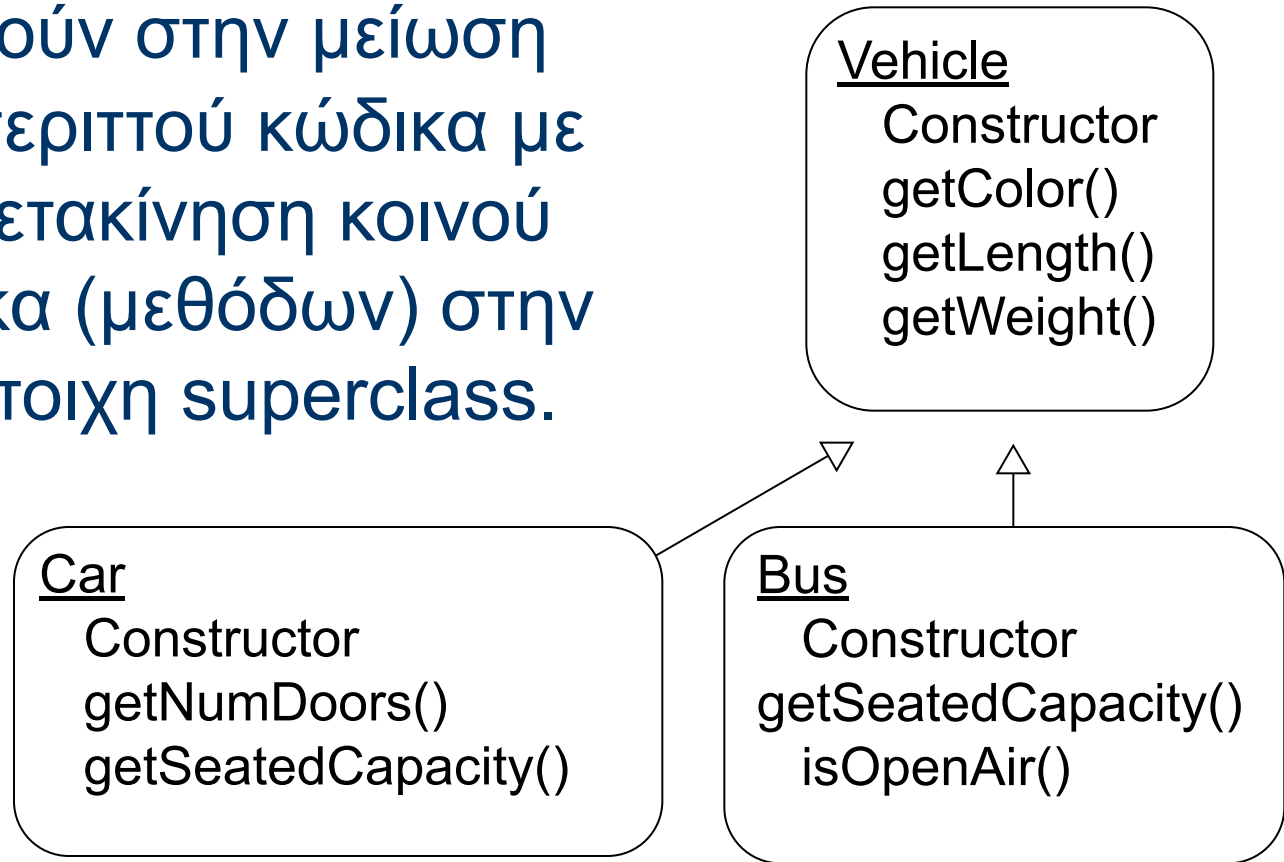
Ιεραρχίες κλάσεων

- Ο προγραμματιστής ορίζει ιεραρχίες κλάσεων
- Οι πιο γενικές κλάσεις βρίσκονται πιο ψηλά προς την κορυφή



Ιεραρχίες κλάσεων (συνέχεια)

- Βοηθούν στην μείωση του περιττού κώδικα με την μετακίνηση κοινού κώδικα (μεθόδων) στην αντίστοιχη superclass.



Πολυμορφισμός (Polymorphism)

- Καθιστά δυνατή την κλήση της σωστής μεθόδου για ένα αντικείμενο ακόμα και όταν το αντικείμενο αναφέρεται ότι ανήκει σε έναν πιο γενικό τύπο (superclass).
- Ο πολυμορφισμός είναι πάντα υπαρκτός στις ιεραρχίες κλάσεων.

A subclass is a specialization of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa. *For example, every circle is a geometric object, but not every geometric object is a circle.*

Πολυμορφισμός (συνέχεια)

Η ενεργή παράμετρος η οποία περνιέται σε αυτή την μεθοδο μπορεί να είναι **Car** ή **Bus**.
—οποιαδήποτε subclass της **Vehicle**.

```
public int calculateCapacity(Vehicle vehicle)
{
    return vehicle.getSeatedCapacity();
}
```

Η σωστή μέθοδος θα κληθεί αυτόματα για κάθε τύπο **Vehicle**.

Abstract κλάσεις

- Κάποιες από τις μεθόδους μιας κλάσης μπορούν να δηλωθούν ως **abstract** και να ορισθούν μόνο **signatures** για αυτές.
- Μία κλάση που περιέχει μία ή περισσότερες **abstract** μεθόδους δηλώνεται ως **abstract**

```
public abstract class Vehicle
{
    ...
    public abstract int getSeatedCapacity();
    ...
    public void move(int distance) { ... }
}
```

Signature
abstract
μεθόδου

Abstract κλάσεις (συνέχεια)

- Οι abstract κλάσεις αποτελούν κοινές superclass για πιο ειδικές κλάσεις
- Οι abstract κλάσεις και μέθοδοι είναι αναγκαίες για την λειτουργία του πολυμορφισμού.
- Οι abstract κλάσεις είναι πιο κοντά στην κορυφή της ιεραρχίας των κλάσεων. Περιγράφουν πιο γενικά αντικείμενα.

Abstract κλάσεις (συνέχεια)

- Η Java δεν επιτρέπει την δημιουργία αντικειμένων abstract κλάσεων
- Μία abstract κλάση όμως μπορεί να έχει constructors οι οποίοι καλούνται από τους constructors των subclass αυτής
- Μία κλάση η οποία δεν περιέχει abstract μεθόδους καλείται *concrete* κλάση. Αν δεν υλοποιεί όλες τις abstract μεθόδους της superclass θα πρέπει και η ίδια να δηλωθεί abstract
- Μία abstract κλάση μπορεί να υλοποιεί ένα υποσύνολο των μεθόδων ενός interface

Η κλάση Object

- Στην Java κάθε κλάση επεκτείνει (extends) την κλάση βιβλιοθήκης Object (στο package java.lang)
- Η Object είναι μία concrete κλάση

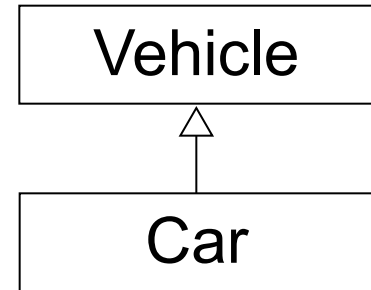
```
public class Object
{
    public String toString {...}
    public boolean equals (Object other) {... }
    public int hashCode() { ... }

    // a few other methods
    ...
}
```

Αυτές οι μέθοδοι ορίζονται ξανά (overridden) με βάση τις ανάγκες των subclass

Η κλάση Object είναι η υπερκλάση όλων των κλάσεων της Java, δεδομένου πως ο compiler την ορίζει αυτόματα ως υπερκλάση σε οποιαδήποτε κλάση δεν κληρονομεί άμεσα από κάποια άλλη.

Κλήση των constructor της superclass



```
public class Car extends Vehicle
{
    // Constructor
    public Car()
    {
        super();
        ...
    }
}
```

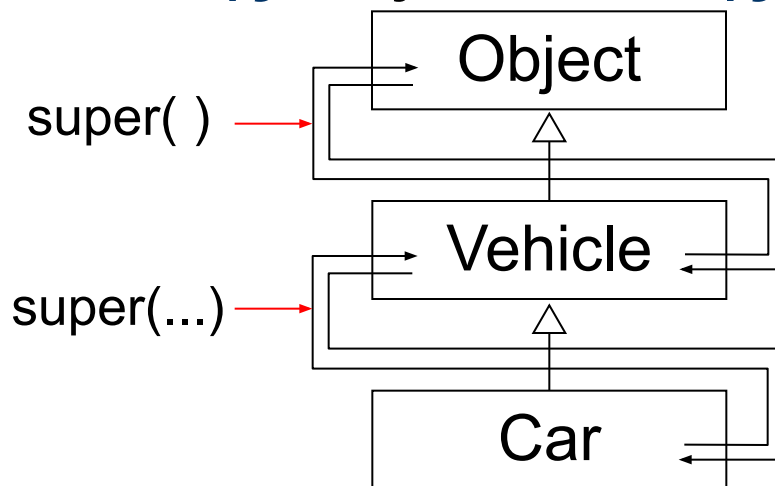
Καλεί τον constructor της **Vehicle**

Ο αριθμός και ο τύπος των παραμέτρων πρέπει να ταιριάζουν με κάποιον από τους constructors της superclass.

Εάν είναι παρούσα, πρέπει να είναι η πρώτη εντολή στον constructor

Κλήση των constructor της superclass (συνέχεια)

- Εάν δεν υπάρχει άμεση κλήση σε κάποιον constructor της superclass, ο no-arg constructor της superclass καλείται έμμεσα.
- Ο constructor της superclass καλεί τον constructor της δικής της superclass μέχρι τον constructor της Object κλάσης.



Polymorphism Demo

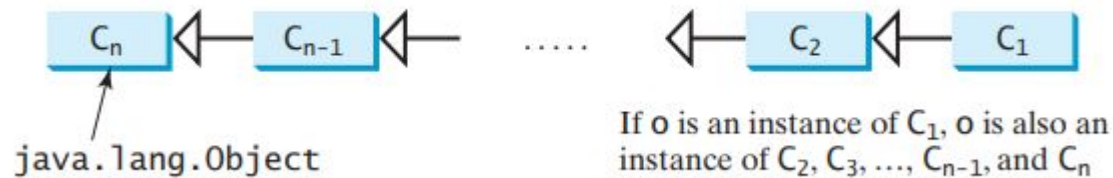
```
public class PolymorphismDemo {  
    /** Main method */  
    public static void main(String[] args) {  
        // Display circle and rectangle properties  
        displayObject(new CircleFromSimpleGeometricObject(1, "red", false)); //polymorphic call  
        displayObject(new RectangleFromSimpleGeometricObject(1, 1, "black", true)); //polymorphic call  
    }  
    /** Display geometric object properties */  
    public static void displayObject(SimpleGeometricObject object) {  
        System.out.println("Created on " + object.getDateCreated() + ". Color is " +  
object.getColor());  
    }  
}
```

```
Created on Mon Mar 09 19:25:20 EDT 2011. Color is red  
Created on Mon Mar 09 19:25:20 EDT 2011. Color is black
```

The method `displayObject` takes a parameter of the `GeometricObject` type. You can invoke `displayObject` by passing any instance of `GeometricObject` (e.g., `new CircleFromSimpleGeometricObject(1, "red", false)` and `new RectangleFromSimpleGeometricObject(1, 1, "black", false)`).

Dynamic binding

- If o (instance of C_1) invokes a method p , the JVM searches for the implementation of the method p in C_1 , C_2 , $\dots$, C_{n-1} , and C_n , in this order, until it is found. Once an implementation is found, the search **stops** and the first-found implementation is invoked.



The JVM dynamically binds the implementation of the method at runtime, decided by the actual type of the variable.

Dynamic binding

```
public class DynamicBindingDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

```
Student  
Student  
Person  
java.lang.Object@130c19b
```

```
class GraduateStudent extends Student {  
}  
class Student extends Person {  
    @Override  
    public String toString() {  
        return "Student" ;  
    } }  
class Person extends Object {  
    @Override  
    public String toString() {  
        return "Person" ;  
    } }
```

When the method `m(Object x)` is executed, the argument `x`'s `toString` method is invoked. `x` may be an instance of `GraduateStudent`, `Student`, `Person`, or `Object`. The classes `GraduateStudent`, `Student`, `Person`, and `Object` have their own implementations of the `toString` method. **Which implementation is used will be determined by `x`'s actual type at runtime.**

Invoking `m(new GraduateStudent())` causes the `toString` method defined in the `Student` class to be invoked

Invoking `m(new Student())` causes the `toString` method defined in the `Student` class to be invoked

Invoking `m(new Person())` causes the `toString` method defined in the `Person` class to be invoked

Invoking `m(new Object())` causes the `toString` method defined in the `Object` class to be invoked

Κλήση μεθόδων της superclass

- **super.** Καλείται από τις subclass μιας superclass (συνήθως χρησιμοποιείται στις subclass των κλάσεων βιβλιοθηκών):

```
public class Canvas extends JPanel
{
    ...
    public void paintComponent (Graphics g)
    {
        super.paintComponent (g);
        ...
    }
    ...
}
```

Interfaces

- Ένα interface είναι όμοιο με μία abstract κλάση, αλλά δεν περιέχει πεδία ή constructors παρά μόνο συμβολικές σταθερές και signatures μεθόδων.

```
public interface Dance
{
    int STEP = 2;
    DanceStep getStep (int i);
    int getTempo ();
    int getBeat (int i);
}
```

- Οι σταθερές εννοείται ότι είναι πάντα public static final ενώ οι μέθοδοι πάντα public abstract. Αυτοί οι modifiers παραλείπονται από το σώμα του interface.

Interfaces (συνέχεια)

- Μία κλάση η οποία υλοποιεί ένα interface πρέπει να δηλωθεί κατάλληλα.
- Μία concrete κλάση η οποία υλοποιεί ένα interface πρέπει να υλοποιήσει όλες τις μεθόδους του interface.

```
public class Waltz implements Dance
{
    ...
    // Methods:
    public DanceStep getStep (int i) { ... }
    public int getTempo () { return 750; }
    public int getBeat (int i) { ... }
    ...
}
```